Especificação Técnica e Governança de Contexto (SDD & AI Harness) — Módulo TaskAnalyzer

Documento de especificação executável, cenários de aceite, diretrizes de governança de IA e arquitetura de repositório referentes à **Fase 1 (Entrega Inicial)** da disciplina Bootcamp III.

1. Visão Geral e Contrato de Negócio (SDD)

1.1 Identificação do Estudante e do Projeto

Campo	Informação do Aluno / Projeto
Nome Completo:	Guilherme Meyer Soares
Curso / Programa:	Análise e Desenvolvimento de sistemas
Polo / Turma:	Turma A
E-mail Institucional:	guigas@sempreceub.com
Disciplina:	Bootcamp III
Versão do Documento:	v1.0 (Contrato Executável - Fase 1)

1.2 Propósito do Módulo TaskAnalyzer

O **TaskAnalyzer** é um módulo avançado de processamento de métricas e análise de produtividade projetado para avaliar o fluxo de execução de tarefas dentro de um sistema de gestão de projetos. No paradigma *Spec-Driven Development (SDD)*, este documento atua como o contrato formal rigoroso e fonte inquestionável da verdade para a geração automatizada de código por agentes de Inteligência Artificial na Fase 2.

Principais objetivos funcionais do módulo:

- **Cálculo do Tempo Médio de Conclusão:** Determinar com precisão a duração média (em horas) gasta para finalizar tarefas, considerando o intervalo entre a data de criação e a data de conclusão.

- **Mapeamento de Taxa de Atraso:** Identificar a proporção de tarefas concluídas após a data limite (prazo) estabelecida, calculando o percentual de estouro de prazo.
- **Indicadores por Prioridade:** Segmentar os dados estatísticos por diferentes níveis de prioridade (Alta, Média e Baixa), permitindo identificar gargalos operacionais em demandas críticas.
- **Suporte à Tomada de Decisão:** Prover dados estruturados e confiáveis para otimização de processos e melhoria contínua da equipe.

1.3 Contrato Executável de Interface

Esta subseção especifica a estrutura rigorosa de dados exigida nas interações com a classe/módulo TaskAnalyzer.

1.3.1 Estrutura dos Dados de Entrada (Inputs):

Campo	Tipo de Dado	Obrigatoriedade	Descrição e Restrições
id	String / UUID	Obrigatório	Identificador único e unívoco da tarefa.
titulo	String	Obrigatório	Título descritivo da tarefa (mínimo de 3 caracteres, não vazio).
prioridade	String (Enum)	Obrigatório	Valores permitidos estritamente: "BAIXA", "MEDIA", "ALTA".
data_criacao	datetime / ISO 8601	Obrigatório	Data e hora de criação no formato YYYY-MM-DDTHH:MM:SSZ.
data_prazo	datetime / ISO 8601	Obrigatório	Data e hora limite estipulada para a conclusão da tarefa.
data_conclusao	datetime / Optional	Opcional	Data de finalização. Nulo (None) para tarefas não concluídas.
status	String (Enum)	Obrigatório	Valores permitidos: "PENDENTE",

Campo	Tipo de Dado	Obrigatoriedade	Descrição e Restrições
			"CONCLUIDA", "CANCELADA".

1.3.2 Estrutura do Resultado da Análise (Outputs / Dicionário de Retorno):

Campo de Saída	Tipo	Descrição e Regra de Cálculo
total_tarefas_analisadas	int	Contagem total de tarefas elegíveis com status "CONCLUIDA".
tempo_medio_conclusao_horas	float	Soma das horas decorridas (data_conclusao - data_criacao) dividida pelo total de concluídas. Arredondado em 2 casas decimais.
taxa_atraso_percentual	float	Porcentagem (0.0 a 100.0) de tarefas concluídas com data_conclusao > data_prazo.
metricas_por_prioridade	dict	Dicionário agrupado pelas chaves "BAIXA", "MEDIA" e "ALTA", contendo para cada uma: total, tempo_medio_horas e taxa_atraso_percentual.

1.3.3 Regras de Negócio e Restrições Mandatórias:

1. **Filtragem de Status:** Apenas tarefas com status "CONCLUIDA" e contendo data_conclusao não nula devem ser contabilizadas no cálculo de médias e taxas de atraso. Tarefas "PENDENTE" ou "CANCELADA" devem ser desconsideradas nos cálculos produtivos.
2. **Invariante Temporal (Validação de Datas):** A data_conclusao não pode ser estritamente anterior à data_criacao. Caso ocorra essa inconsistência em qualquer registro fornecido, a execução deve ser interrompida imediatamente disparando uma exceção tratada do tipo ValueError com a mensagem: "Data de conclusão não pode ser anterior à data de criação da tarefa.".
3. **Prevenção de Divisão por Zero:** Se a lista de entradas não mantiver nenhuma tarefa

concluída válida (ou for totalmente vazia), o sistema não deve interromper por erro genérico de divisão por zero. Ele deve retornar um dicionário com métricas zeradas (total_tarefas_analisadas = 0, tempo_medio_conclusao_horas = 0.0, taxa_atraso_percentual = 0.0) e emitir um log de alerta (logging.warning).

2. Especificação de Cenários de Aceite e Test Harness

2.1 Cenários de Aceite Formalizados (Dado - Quando - Então)

Cenário 1: Sucesso - Processamento de Tarefas Válidas e Métricas Precisas

Dado um conjunto de 3 tarefas com dados válidos:

- Tarefa 1 (ALTA): Criada em 2026-03-01T08:00:00, Prazo 2026-03-01T12:00:00, Concluída em 2026-03-01T10:00:00 (Duração: 2.0h, Sem Atraso). Status: CONCLUIDA.
- Tarefa 2 (ALTA): Criada em 2026-03-01T08:00:00, Prazo 2026-03-01T10:00:00, Concluída em 2026-03-01T12:00:00 (Duração: 4.0h, Com Atraso). Status: CONCLUIDA.
- Tarefa 3 (MEDIA): Criada em 2026-03-01T08:00:00, Prazo 2026-03-02T08:00:00, Sem data de conclusão. Status: PENDENTE.

Quando a função TaskAnalyzer.calcular_metricas(tarefas) for executada,

Então o resultado retornado deve conter rigorosamente:

- total_tarefas_analisadas igual a **2** (ignorando a tarefa PENDENTE).
- tempo_medio_conclusao_horas igual a **3.0** horas ((2.0 + 4.0) / 2).
- taxa_atraso_percentual igual a **50.0%** (1 tarefa atrasada de 2 concluídas).
- No subagrupamento de prioridade "ALTA": total = 2, tempo médio = 3.0h, taxa de atraso = 50.0%.

Cenário 2: Exceção / Erro - Tratamento de Datas Inválidas e Inconsistência

Dado uma tarefa com registro de datas inconsistente:

- Tarefa Invalida: Data de Criação = 2026-03-02T10:00:00 e Data de Conclusão = 2026-03-01T10:00:00 (Conclusão anterior à criação). Status: CONCLUIDA.

Quando o analisador de tarefas for executado com essa entrada,

Então o sistema deve interromper a execução e disparar uma exceção específica ValueError contendo mensagem explicativa clara tratada, registrando um log de erro no padrão logging.error sem crash não tratado da aplicação.

2.2 Planejamento do Test Harness

O **Test Harness** consiste no arcabouço de testes automatizados desenvolvido em pytest que

atuará na Fase 2 como um "ambiente de teste isolado" e validador automático do código escrito pelo assistente de IA.

- **Automatização BDD:** Cada cenário de aceite especificado na Seção 2.1 será traduzido em rotinas de teste unitário no arquivo tests/test_harness.py.
- **Execução de Suíte Completa:** Durante a Fase 2, todo código gerado por agentes de IA só será aceito se passar 100% sem falhas na suíte do Test Harness.
- **Garantia contra Regressões:** Permite validar se ajustes e otimizações efetuados pela IA mantêm a estrita aderência ao contrato executável de negócio.

3. Governança de Contexto e Regras para Agentes de IA (CONTEXT_RULES)

Para garantir que o assistente de IA produza código seguro, padronizado, legível e livre de alucinações durante a Fase 2, o desenvolvimento será regido pelo conjunto de regras persistentes contidas abaixo, que farão parte do arquivo CONTEXT_RULES.md no repositório.

3.1 Diretrizes Arquiteturais (Obrigatórias)

- **Versão da Linguagem:** Utilização estrita do Python 3.11 ou superior como versão mínima obrigatória.
- **Type Hints Totais:** Obrigatoriedade do uso de anotações de tipo explícitas (Type Hints) em todos os parâmetros, variáveis públicas e retornos de funções/métodos.
- **Padrões de Código e Clean Code:** Conformidade rigorosa com a guia de estilo PEP 8 e aplicação do Princípio da Responsabilidade Única (SRP - Single Responsibility Principle).
- **Documentação Formal:** Utilização do padrão *Google Style Docstrings* em todos os módulos, classes, métodos e exceções.
- **Modularidade:** As funções devem ser pequenas, coesas, altamente legíveis e com um único propósito bem definido.
- **Tratamento Estruturado de Erros:** Disparo de exceções específicas e tratadas (ex: ValueError, KeyError), evitando blocos genéricos de except Exception: pass.
- **Observabilidade e Logs:** Uso obrigatório da biblioteca padrão logging configurada em formato estruturado (INFO para fluxos normais, WARNING para listas vazias, ERROR para dados inconsistentes).

3.2 Proibições Explícitas (Regras Restritivas)

- **Proibição de Bibliotecas Externas Não Autorizadas:** É proibido utilizar qualquer biblioteca

externa de terceiros não listada formalmente no requirements.txt (permitido apenas o ecossistema padrão e pytest).

- **Inalterabilidade dos Testes:** É terminantemente proibido alterar, remover, flexibilizar ou burlar os cenários de teste fornecidos no Test Harness.
- **Respeito à Arquitetura:** É proibido modificar a estrutura de pastas e arquivos definida na especificação do repositório.
- **Sem Persistência Lateral:** É proibido persistir dados em banco de dados, arquivos locais (ex: CSV, JSON no disco) ou serviços externos. O processamento deve ocorrer estritamente em memória.
- **Imutabilidade de Assinaturas:** É proibido alterar os nomes e parâmetros das funções públicas estipuladas no contrato executável sem autorização expressa.
- **Geração com Testes:** Proibido criar ou sugerir novas funcionalidades de código sem disponibilizar os testes unitários equivalentes.
- **Sem Código Duplicado:** Proibido gerar trechos redundantes, código morto (*dead code*) ou comentários excessivos e óbvios.
- **Sem Suposições não Especificadas:** A IA não deve assumir comportamentos de negócio que divirjam ou extrapolem o contrato especificado.

3.3 Regras de Interação com a IA

Sempre que um prompt for enviado a um agente de IA para geração ou modificação do módulo TaskAnalyzer, a instrução deve conter obrigatoriamente como prefixo o arquivo CONTEXT_RULES.md na íntegra. Respostas que descumprirem qualquer uma das proibições devem ser sumariamente rejeitadas pelo desenvolvedor.

4. Arquitetura do Repositório e Preparação para Git/GitHub

4.1 Desenho da Árvore do Repositório (Fase 2)

Abaixo está apresentada a estrutura completa de arquivos do repositório sdd-task-analyzer que será criada e versionada no GitHub durante a Fase 2:

```
sdd-task-analyzer/
├── README.md                  # Visão geral do projeto, instruções de execução e documentação do repositório
├── CONTEXT_RULES.md           # Arquivo de regras persistentes e governança para os agentes de IA
├── specs/
│   └── task_analyzer_spec.md # Especificação técnica SDD detalhada (base desta Fase 1)
├── tests/
│   └── test_harness.py        # Testes automatizados com pytest para validação do código
├── src/
```

```
|   └── task_analyzer.py      # Código-fonte principal gerado via IA e homologado
pelo desenvolvedor
├── .gitignore                # Regras de arquivos/pastas ignorados pelo
versionamento Git
└── requirements.txt          # Dependências do ambiente Python (ex: pytest)
```

4.2 Plano de Homologação Humana

O papel do desenvolvedor no paradigma SDD é o de arquiteto e homologador crítico. Todo código gerado por IA na Fase 2 passará obrigatoriamente pelas seguintes etapas de validação antes de ser incorporado à branch principal:

1. **Execução Automatizada do Test Harness:** Rodar pytest e garantir 100% de aprovação e cobertura nos cenários descritos na especificação.
2. **Auditoria de Conformidade e Regras:** Verificar se a solução não violou nenhuma regra restritiva estipulada em CONTEXT_RULES.md (uso de libs não autorizadas, alterações de contrato, etc.).
3. **Análise Qualitativa de Código:** Avaliar legibilidade, clareza dos nomes, aderência ao PEP 8, corretude dos type hints e formatação de docstrings.
4. **Testes Manuais de Borda:** Executar testes com valores nulos, entradas atípicas e volume elevado de registros para identificar alucinações não previstas.
5. **Homologação e Versionamento:** Realizar o commit e autorizar o Pull Request apenas após aprovação formal do homologador humano.

4.3 Estratégia de Versionamento e Git Flow (Fase 2)

Será adotado um modelo Git Flow simplificado. A branch main conterá o código estável e homologado. O desenvolvimento das funcionalidades geradas por IA ocorrerá em branches do tipo feature/nome-da-feature, que serão submetidas via **Pull Request (PR)** para revisão crítica e aprovação final.

5. Requisitos e Roteiro do Vídeo de Apresentação

5.1 Checklist de Requisitos Obrigatórios do Vídeo

- **Apresentação em Câmera:** Aparição obrigatória do estudante na câmera (mostrando o rosto claramente) com áudio inteligível.
- **Duração Fixa:** Exatamente 5 minutos de gravação.
- **Formato da Gravação:** Formato horizontal padrão (proibido o envio de vídeos em formato YouTube Shorts / vertical).
- **Plataforma e Visibilidade:** Publicado no YouTube na visibilidade "Público" ou "Não

Listado".

- **Link do Vídeo no Documento:** O link do vídeo deve ser colado abaixo neste documento e no formulário do AVA.

5.2 Roteiro Sequencial da Apresentação (5 Minutos)

1. **Minuto 1: Apresentação Pessoal:** Nome completo, curso, polo e motivação de aprendizagem no Bootcamp III com SDD.
2. **Minutos 2 e 3: Apresentação da Especificação SDD:** Exibir este documento no Google Docs, explicando o contrato de negócio, entradas, saídas e os cenários de aceite (Dado-Quando-Então).
3. **Minuto 4: Regras de Governança de IA:** Explicar as diretrizes contidas em CONTEXT_RULES.md e como elas evitam alucinações da IA.
4. **Minuto 5: Estrutura do Repositório e Conclusão:** Detalhar a árvore de arquivos planejada para o GitHub e encerrar a gravação.

5.3 Links do Entregável

Item	Link de Acesso
Link do Vídeo no YouTube:	https://youtu.be/TLJbFApSkV4